

Scott Curtis
WGU Email: scurt63@wgu.edu
7/3/2025
Program Mentor: DeNece Meyer

NHP3 — NHP3 Task 2: WGUPS Routing Program Implementation (Writing Portion: F- I)

F1. Describe two or more strengths of the algorithm used in the solution:

The two defining strengths of the Nearest Neighbor Algorithm implemented for this project are *simplicity* and *speed*.

Simplicity: It is simple to implement, easy to adapt to new use cases and parameters, and perfect for small datasets like the scenario defined in the project requirements. Sometimes, simplicity is imperative for small scale companies and projects without large development teams, resources, and timeframes. In my opinion, NNA is the easiest algorithm to implement for the project context.

Speed: While compatibility for the project requirements and ease of implementation are its combined number one strength, the Nearest Neighbor is also quite fast, providing quick, greedy solutions that run at excellent speeds for the project's 40 package address dataset. It provides speeds of $O(n^2)$ for n packages, or nodes. For the size and scale of this project, it is faster and more efficient than Dijkstra's or Genetic Algorithm.

F2. Verify that the algorithm used in the solution meets *all* requirements in the scenario:

The implemented algorithm ensures all WGUPS packages are delivered on time within the constraints of the project scenario guidelines, as demonstrated by the provided screenshots of the simulation output results, submission of all code, and by thorough and frequent author test runs.

F3. Identify two other named algorithms that are different from the algorithm implemented in the solution and would meet *all* requirements in the scenario:

Dijkstra's Algorithm: This was a strong second choice for the project, if I couldn't have chosen Nearest Neighbor. I chose Nearest Neighbor for its ease of implementation and speed with small inputs, but Dijkstra's would have been an excellent choice, especially if mileage constraints were greater than 140 and the dataset was larger. Rather than making greedy local decisions, it provides the guaranteed best route between stops, which would give a much greater long run optimum than NNA.

Genetic Algorithm: Genetic Algorithms use operations like fitness evaluation, selection, crossover, and mutation, to quite literally simulate the evolutionary processes of natural selection to create a metaheuristic approach, which means it evolves solutions over “generations,” combining and mutating different approaches. For this project’s scope, this would mean simulating many routes and blending them to find the best combination and route strategy, becoming as close to a perfectly optimal global solution as possible. Due to complexity, it wouldn’t be my first, nor second choice, but if massive project scaling and global optimization were necessary for huge datasets, like those of Amazon or UPS or another large delivery company, it would make an excellent option. It meets all project requirements.

F3a. How both algorithms differ from Nearest Neighbor:

Dijkstra’s Algorithm: This algorithm differs from the chosen algorithm because while the nearest neighbor makes greedy decisions based on the next closest node and favors a local, immediate best path, Dijkstra’s heuristics favor a total trip/global optimum, often choosing immediate paths that are not necessarily the closest to the current location. It is more globally optimal than NNA and outperforms it for large datasets, while taking a massive hit in performance and speed for smaller datasets.

Genetic Algorithm: Similar to Dijkstra’s, Genetic Algorithms differ from nearest neighbor by pursuing a globally optimal result over a greedy, locally optimal one. This obviously produces much better outcomes for large datasets with huge quantities of nodes, but is less efficient, slower, and has worse computational performance when used on small quantities of nodes, where a global optimum might make a very minimal difference, and the computational intensiveness of these more complex algorithms is entirely unnecessary. In lesser words, Genetic Algorithms would be overkill for the scope of the project.

G. Describe what you would do differently, other than the two algorithms identified in part F3, if you did this project again, including details of the modifications that would be made:

If I were to do this project again, I would experiment with a more complex algorithm, just for fun, that is closer to the ones Amazon, UPS, and other international supply chain and logistic enterprise companies use. While I firmly believe the Nearest Neighbor was best for the project scenarios and requirements, I would have enjoyed implementing another more complex algorithm for the challenge and experience. I might have chosen a genetic algorithm just for the immersion.

Additionally, if I had more time, or a second try, I would implement meticulous unit testing, and lots of it. While I did use frequent bug testing, I think more intentional and repeatable unit testing is an invaluable skill to have as a programmer and I would have loved to incorporate testing classes in my project to apply some of the principles I have

learned in past classes, such as Software Quality Assurance. Not only is this considered best practice, but it would have saved me many headaches along the way.

H. Verify that the data structure used in the solution meets *all* requirements in the scenario.

The custom hash table used in this project meets all requirements in the project scenario. It supports constant-time insertion and retrieval of package data, allowing lookups, updates, and reporting of packages statuses at any chosen time. It does not incorporate third party libraries for parsing or hashing, beyond python's hash() function for the hash key, and has been adapted to be dynamic through logic and a resize() method within the hash table.

H1. Identify two other data structures that could meet the same requirements in the scenario.

AVL Tree: An AVL Tree, named after Adelson-Velsky and Landis, is a self-balancing binary search tree. It automatically rotates nodes after all insertions and deletions to keep the tree balanced (the height of the left and right subtrees differ by never more than 1). This guarantees log n time complexity across these operations, no matter the size of the input. This would have been the optimal data structure if datasets were huge.

Splay Tree: A splay tree is a self-adjusting binary search tree. Whenever a node is accessed, it moves it to the root through "splay" rotations. This makes access to recently utilized nodes faster. Splay trees do not guarantee perfect balance like the AVL, but are more adaptive for frequently repeated patterns.

H1a. Describe how *each* data structure identified in H1 is different from the data structure used in the solution.

AVL Tree: AVL trees are balanced trees, which means that they keep the same order and data is always sorted and balanced. This is unlike the unordered hashtable, which makes search, insert, or delete operations efficient even with large datasets. However, unlike the hash table, which operates in an average time complexity of constant time, AVL trees operate in $O(\log n)$ time for search, insert, and delete operations. This is much slower than the hash table for small datasets, and would not be useful for this context unless ranged queries or sorted traversals were necessary. AVL trees are faster than hash tables in Big O worst case time and at large datasets, but in applications of limited size like this one, it is very simple to keep hash tables running at constant time through regulation of their load factors and rehashing when necessary.

Splay Tree: Splay trees adapt to access patterns, and the most frequently or recently accessed items are faster. Hash tables differ from this because access is always out of order. Over the long run, most splay lookups are fast with a time complexity of $O(\log n)$, but splay trees are not strictly balanced like the AVL tree, so some single operation lookups can potentially take $O(n)$, which is significantly slower than the hash table for the scope of this project.

I. Acknowledge sources, using in-text citations and references, for content that is quoted, paraphrased, or summarized.

Adelson-Velsky, Georgy , and Evgenii Landis. *An Algorithm for the Organization of Information*. 1960.

Albers, S. "Randomized Splay Trees: Theoretical and Experimental Results." *Information Processing Letters*, vol. 81, no. 4, 28 Feb. 2002, pp. 213–221,
[https://doi.org/10.1016/s0020-0190\(01\)00230-7](https://doi.org/10.1016/s0020-0190(01)00230-7). Accessed 18 Feb. 2020.

WGU CS Faculty. *ZyBooks*.

<https://Learn.zybooks.com/Zybook/WGUC950Template2023/Chapter/4/Section/3>
, Western Governors University. (2023). Data Structures and Algorithms II (C950)., 2023.